OOP in C++

Template

Instructor: Dinesh Maharjan

2022

Why Template

```
#include<iostream>
using namespace std;
int compare(int a,int b){
	if(a>b){
		return a;
	}
	else{
		return b;
	}
}
```

```
float compare(float a,float b){
	if(a>b){
		return a;
	}
	else{
		return b;
	}
}
```

Why Template

```
int main(){
        float a=55.4;
        float b=445.99;
        int c=44;
        int d=99;

cout<<compare(a,b)<<endl;

cout<<compare(c,d)<<endl;
        return 0;
}
```

Suppose that…

- You want to write a function to compare two ints
- You want to write a function to compare two strings
  - Function overloading!

```
// returns 0 if equal, 1 if value1 is bigger, -1 otherwise
int compare(const int &value1, const int &value2) {
  if (value1 < value2) return -1;
  if (value2 < value1) return  1;
  return 0;
}

// returns 0 if equal, 1 if value1 is bigger, -1 otherwise
int compare(const string &value1, const string &value2) {
  if (value1 < value2) return -1;
  if (value2 < value1) return  1;
  return 0;
}
```

Hm…

- The two implementations of `compare` are nearly identical!
  - What if we wanted a version of `compare` for *every* comparable type?
  - We could write (many) more functions, but that's obviously wasteful and redundant
- What we'd prefer to do is write "*generic code*"
  - Code that is type-independent
  - Code that is compile-type polymorphic across types

- C++ templates were originally designed to reduce duplication of code

- instead of making functions for each type e.g. *float and double*

```
float
distance(float a1, float a2, float b1, float b2)
  {
  float tmp1 = a1 - b1;
  float tmp2 = a2 - b2;

  return std::sqrt( tmp1*tmp1 + tmp2*tmp2 );
  }
```

```
double
distance(double a1, double a2, double b1, double b2)
  {
  double tmp1 = a1 - b1;
  double tmp2 = a2 - b2;

  return std::sqrt( tmp1*tmp1 + tmp2*tmp2 );
  }
```

- we can define a function template to handle both *float* and *double*

```
template <typename T>
T
distance(T a1, T a2, T b1, T b2)
  {
  T tmp1 = a1 - b1;
  T tmp2 = a2 - b2;

  return std::sqrt( tmp1*tmp1 + tmp2*tmp2 );
  }
```

- so we've saved ourselves repeating code -> less bugs!
- but! the template actually allows more than just *float* and *double*...
- you can feed it a wrong type by accident -> more bugs!
- we will come back to this

- templates can also be used to implement programs that are run at *compile time*

- why would you ever want to do that ??

- example: compute the factorial function, noted as “n!”

- product of a positive integer multiplied by all lesser positive integers, eg. 4! = 4 * 3 * 2 * 1

- **traditional implementation:**

```
int factorial(int n)
  {
  if (n==0)                          // terminating condition
    return 1;
  else
    return n * factorial(n-1);       // recursive call to factorial()
  }

void user_function()
  {
  int x = factorial(4);              // 4 * 3 * 2 * 1 * 1 = 24
  }
```

Template

The C++ template mechanism allows a type to be a parameter in the definition of a class or a function.

- Function template
- Class template

Template provides a powerful mechanism for code reuse.

Function Template

To make a function general purpose, we can write a function template with some unspecified generic form of data element.

Function template $\Longrightarrow$ function with type parameter.

C++ Templates

- A C++ template is, well, a generic code template from which specific code can be generated
  - The template takes one (or more) arguments that typically are type names
  - When the compiler encounters a use of the generic code, it instantiates the specific concrete realization of it according to the types the generic code is being asked to operate on
- How does the compiler determine the types for a particular use?
  - The programmer tells it, or
  - The compiler can figure it out because it knows the types of the arguments

C++ Function Templates

```
#include <iostream>
#include <string>

// returns 0 if equal, 1 if value1 is bigger, -1 otherwise
template <typename T>    // <...> can also be written <class T>
int compare(const T &value1, const T &value2) {
  if (value1 < value2) return -1;
  if (value2 < value1) return  1;
  return 0;
}

int main(int argc, char **argv) {
  std::string h("hello"), w("world");
  std::cout << compare<int>(10, 20) << std::endl;
  std::cout << compare<std::string>(h, w) << std::endl;
  std::cout << compare<double>(50.5, 50.6) << std::endl;
  std::cout << compare <int>(-5, -20) << std::endl;
  return 0;
}
```

No code is generated for the template

Three programmer explicit instantiations

Compiler Inference

- Sometimes the compiler can figure out the instantiated types itself

```
#include <iostream>
#include <string>

// returns 0 if equal, 1 if value1 is bigger, -1 otherwise
template <typename T>
int compare(const T &value1, const T &value2) {
  if (value1 < value2) return -1;
  if (value2 < value1) return  1;
  return 0;
}

int main(int argc, char **argv) {
  std::string h("hello"), w("world");
  std::cout << compare(10, 20) << std::endl; // ok
  std::cout << compare(h, w) << std::endl;   // ok
  std::cout << compare("Hello", "World") << std::endl;  // hm...
  return 0;
}
```

Yeah! ?

What's Going On?

- The compiler doesn't generate any code when it sees the template function
  - It doesn't know what code to generate yet, since it doesn't know what types are involved
- When the compiler sees the function being used, then it understands what types are involved
  - It generates the ***instantiation*** of the template and compiles it (kind of like macro expansion)
    - The compiler generates template instantiations for *each* type used as a template parameter

A simple example:

```
// for integer type
int largerOfTwo( int x, int y ){
   return x > y ? x : y;
}

// for double type
double largerOfTwo( double x, double y ){
   return x > y ? x : y;
}

// for char type
char largerOfTwo( char x, char y ){
   return x > y ? x : y;
}
```

A function is a candidate to be made a template when its implementation remains unchanged over a set of instances, each of instance handles a unique data type.

Function Template

```
#include<iostream>
using namespace std;

template<typename D>
D maxx(D a,D b){
	if(a>b){
		return a;
	}
	else{
		return b;
	}
}
```

```
template<typename D>
D small(D a,D b){
	if(a<b){
		return a;
	}
	else{
		return b;
	}
}
```

Function Template

```
int main(){
        float a=55.4;
        float b=445.99;
        int c=44;
        int d=99;
        cout<<maxx<float>(a,b)<<endl;
        cout<<small(c,d)<<endl;
        return 0;
}
```

Function Template Overloading

```
#include<iostream>
using namespace std;

template<typename D>
void display(D a){
        cout<<a<<endl;
}
template<class D>
void display(D a[], int n){
        for(int i=0;i<n;i++)
                cout<<a<<endl;
}
```

```
int main(){
        float a=55.4;
        int c=10;
        int b[]={2,3,4};
        display(a);
        display(c,5);
        return 0;
}
```

Array in Function Template

```
#include<iostream>
using namespace std;

template<typename D>
D maxx(D a[],int n){
	D mx=a[0];
	for(int i=0;i<n;i++){
		if (a[i]>mx){
			mx=a[i];
		}
	}
	return mx;
}
```

```
int main(){
	int c[3]={2,3,4};
cout<<maxx(c,3)<<endl;
	return 0;
}
```

Instead of writing different versions of `largeOfTwo` for different data types (function overloading), we define a template function:

```
template< class T >
T largerOfTwo( T x, T y ){
  return x > y ? x : y;
}
```

or

```
template< typename T >
T largerOfTwo( T x, T y ){
  return x > y ? x : y;
}
```

- A template function starts with keyword `template`.
- `T` is a template **type parameter**. It is a symbol serves as a placeholder for a specific data type.
- Keyword `class` and `typename` have the same meaning. They are used to declare different template type parameters.

Function Template Instantiation

Now, we can **instantiate** the template function $\Longrightarrow$ to construct individual functions given a set of actual types or values.

```
template< typename T >
T largerOfTwo( T x, T y ){
   return x > y ? x : y;
}

int main(){
   int x1 = 0, y1 = 1;
   cout << ``the larger one is: ''
        << largerOfTwo( x1, y1 ) << endl;
}
```

When a template function is called, the actual parameters passed to the function are used to determines which version of the template function is constructed. That is, the template type parameters are **deduced** from the function's actual parameters.

Different versions of `largerOfTwo` are generated by the compiler given different types of parameters.

```
int main(){
    int x1 = 0, y1 = 1;
    double x2 = 0.8, y2 = 8.9;
    char x3 = 'c', y3 = 'z';

    cout << "The larger ones are: \n";

    // int version generated
    cout << largerOfTwo( x1, y1 ) << "\n";

    // double version generated
    cout << largerOfTwo( x2, y2 ) << "\n";

    // char version generated
    cout << largerOfTwo( x3, y3 ) << "\n";
 }
```

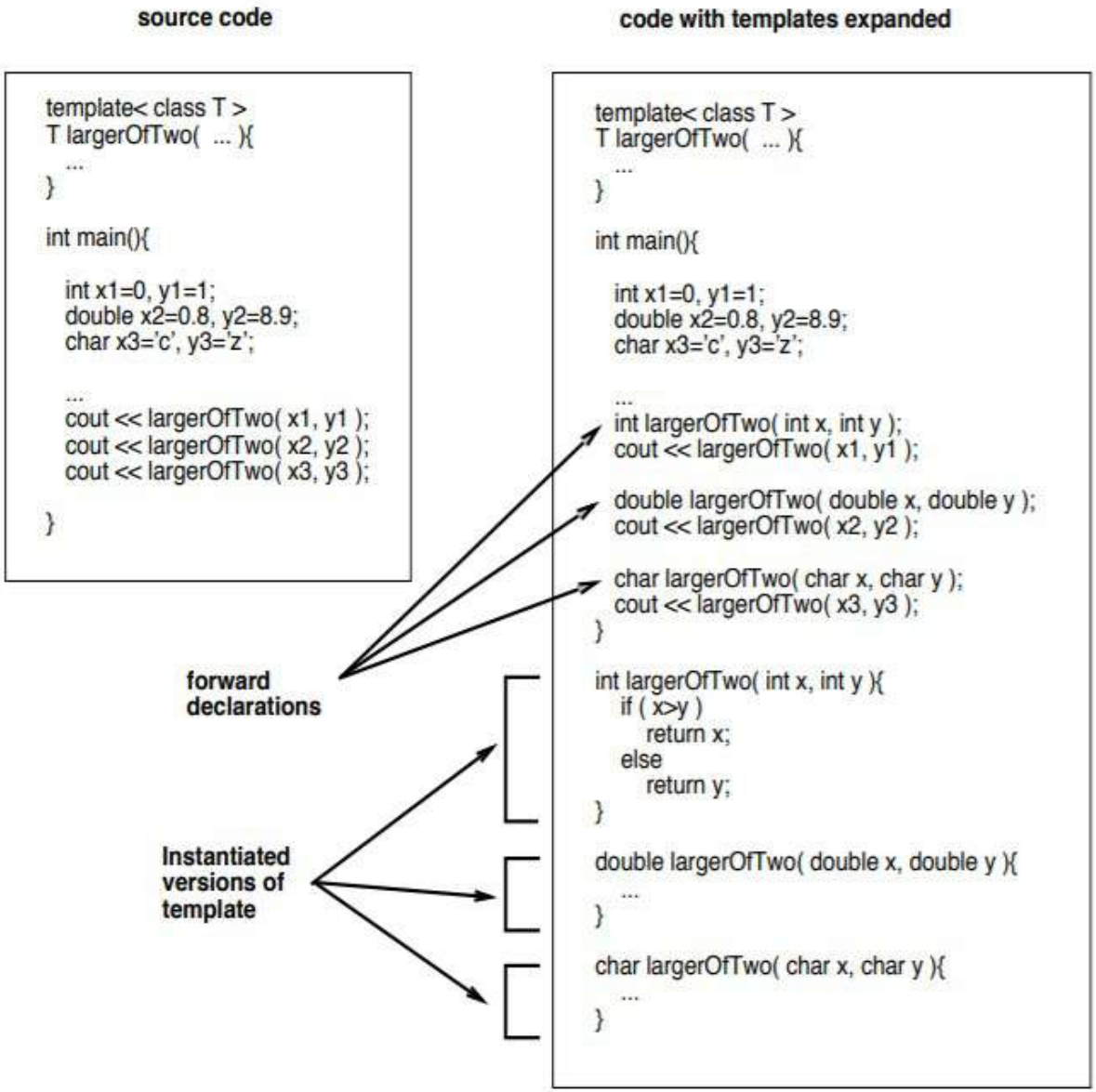


Illustration of template instantiation.

A template can have non-type parameters. A non-type parameter indicates that the parameter name represents a constant value in the template definition.

Example:

```
template <typename T, int size>
T max( T (&array)[size] ) {
  T max_val = array[0];
  for ( int i=1; i<size; i++ )
  if ( array[i] > max_val )
      max_val = array[i];
  return max_val;
}
```

If a function template has more than one template type parameter, each type parameter must be preceded by keyword `class` or `typename`.

Example 1:

```
#include <iostream>
using namespace std;

template <typename T>
T max( T *array, int size ) {
  T max_val = array[0];
  for ( int i=1; i<size; i++ )
    if ( array[i] > max_val )
      max_val = array[i];
  return max_val;
}

int main(){
  int a[5] = { 1, 4, 6, 7, 9 };
  double b[4] = { 2.3, 10.5, 34.7, 11.2 };

  cout << "max of int array is : " << max( a, 5 ) << endl;
  cout << "max of double array is : " << max( b, 4 ) << endl;
}
```

Example 2:

```
#include <iostream>
using namespace std;

template <typename T, int size>
T max( T (&array)[size] ) {
  T max_val = array[0];
  for ( int i=1; i<size; i++ )
    if ( array[i] > max_val )
      max_val = array[i];
  return max_val;
}

int main(){
  int a[] = { 1, 4, 6, 7, 9 };
  double b[4] = { 2.3, 10.5, 34.7, 11.2 };

  cout << "max of int array is : " << max( a ) << endl;
  cout << "max of double array is : " << max( b ) << endl;
}
```

Class Template

Class template in C++ provides a powerful mechanism for code reusability especially in the implementation of data structures (vectors, lists, trees, etc.)

class template $\Longrightarrow$ class with type parameter

- …and class templates

```
template <class T>
class Foo {
// declarations
   bool contains(T &item);
};

template <class T>
bool Foo<T>::contains(T &item)
{ //definition using T }
```

Class Template

```
template<typename D>
class Stack{
    D data[SZ];
    int top;
    public:
    Stack(){
        top=-1;
    }
    void push(D item){
        if(top==SZ-1)
            throw "Stack is full";
        data[++top]=item;
    }
    D pop(){
        if (top==-1)
            throw "Stack is empty";
        return data[top--];
    }
};
```

```
int main(){
    Stack<int> s;
    try{
        s.push(5);
        s.push(10);
        s.push(100);
        s.push(55);
        cout<<s.pop()<<endl;
        cout<<s.pop()<<endl;
    }catch(const char *e){
        cout<<e<<endl;
    }
    return 0;
}
```

Class Set

```
#include<iostream>
#include<set>
using namespace std;

int main(){
    set<int> s;
    s.insert(5);
    s.insert(6);
    s.insert(9);
    s.insert(9);
    for(int e:s)
        cout<<e<<endl;
    return 0;
}
```

Class Set

```
int main(){
    set<int> s;
    s.insert(5);
    s.insert(6);
    s.insert(9);
    s.erase(9);
    for(int e:s)
        cout<<e<<endl;
   Set<int>::iterator it;
   it=s.begin();
   cout<<*it<<endl;
   if(s.find(4)!=s.end())
          cout<<”Set contains 4”<<endl;
    return 0;
}
```

Set Intersection

```
set<int> getIntersection(set<int> s1,set<int>s2){
    set<int>temp;
    set<int>intersection;
    if(s1.size()<s2.size()){
        temp=s1;
        s1=s2;
        s2=temp;
    }
    for(int e:s1)
        if(s2.find(e)!=s2.end())
            intersection.insert(e);
    return intersection;
}
```

Class Vector

```
#include<iostream>
#include<vector>
using namespace std;

int main(){
    vector<int> s;
    s.push_back(5);
    s.push_back(6);
    s.push_back(9);
    s.push_back(9);
    for(int e:s)
        cout<<e<<endl;
   s.erase(s.begin()+1);//delete second element
    return 0;
}
```

Class Map

```
int main(){
    map<string,int> s;
    s["pat"]=1;
    s["rick"]=2;
    s["hari"]=3;
    for(pair<string,int> e:s)
        cout<<e.first<<e.second<<endl;

   map<string,int>::iterator it;
    it=s.begin();
    cout<<it->first<<it->second<<endl;
    s.erase("pat");
    it=s.find("pat");
    if(it!=s.end())
        cout<<"Map contains pat"<<endl;
    return 0;
}
```

OOP in C++

Exception Handling

Instructor: Dinesh Maharjan

2022

Error and Exception

- Errors are the events that does not allow to compile the program
- It terminates program abnormally.
- It is basically two types.
- Compile time error
- Run time error (Logical error)
-

Error and Exception

- **Compile time error:**
- It arise when rules of language are not followed.
- They are detected by compiler while compiling.
- Compiler report about the error with line numbers.
- So, there is no worry about compile time error.
- We can easily correct compile time error.
- This type of error cannot be handled or bypassed.
- It does not harm much on as we cannot run the program.
- Int a
- x=(x+5
- Int a=”hello”;

Error and Exception

- **Run time error:**
- Run time errors are the events that make running program to terminate abnormally.
- It occurs in the course of running a program.
- It can be handled or bypassed so that program terminates normally.
- If it is not handled than resource will be wasted.
- It can generated random results or erroneous result.
- Many situations can cause run time error.
- In c++, run time error is known as exception.

```
int main(){
        cout<<”Hello”<<endl;
        cout<<5/0<<endl;
        cout<<”World”<<endl;
        return 0;
}
```

When program terminate abnormally and show unexpected behavior.

```
int main(){
        cout<<sqrt(-12)<<endl; //nan
        cout<<”Hello”<<endl;
        return 0;
}
```

C++ compiler outputs nan and prints hello

Causes of Exception

- Inputting invalid data
- Dividing by zero.
- Accessing an element out of bounds of an array.
- Low free memory.
- Low Disk space.
- Pushing to a full stack.
- Poping from empty stack.
- Arithmetic overflow, underflow
- Square root of negative number.

Handling Exception

- To handle exception, we need to use
- try block
- catch block, and
- throw block
-
- try block must be followed by catch block.
- Any code that can generate exception must be kept inside the try block.
- But before that code, we need to check if the code generates exception.
- If it generates exception, the throw that exception.
- Functions also can generate exception.
- So such functions must be called inside the try block.

Exception in Main

```
#include<iostream>
using namespace std;
int main(){
    int a=10;
    int b=0;
    try{
        if(b==0)
            throw "Division by negative number";
        cout<<a/b<<endl;
    }catch(const char *e){
        cout<<e<<endl;
    }

    cout<<"Hello"<<endl;
    return 0;
}
```

Multiple Exceptions

```
#include<iostream>
using namespace std;
int main(){
    int a;
    cout<<"Enter any
integer"<<endl;
    cin>>a;
    try{
        if(a<0)
            throw "Negative Number";
        else if (a<3)
            throw a;
        else if (a<6)
            throw float(a);
        else
            throw double(a);
    }
```

Multiple Exceptions

```
catch(const char *e){
        cout<<e<<endl;
    }catch (int e){
        cout<<e<<endl;
    }catch(float e){
        cout<<e<<endl;
    }catch(...){
        cout<<"Default
exception"<<endl;
    }
    return 0;
}
```

Exception in Function

```
#include<iostream>
#include<cmath>
using namespace std;

float compute(int n){
        if(n<0){
                throw "Negative
number does not have root";
        }
        return sqrt(n);
}
                                int main(){
                                        int a;
                                        cout<<"Enter any integer"<<endl;
                                        cin>>a;
                                        try{
                                                float sq=compute(a);
                                                cout<<"Square root "<<sq<<endl;
                                        }
                                        catch (const char *e){
                                                cout<<e<<endl;
                                        }

                                        return 0;}
```

Exception in Class

```
class Calculator{
    double res=0.0;
    public:
    void division(){
        double a,b;
        cout<<"Enter numbers"<<endl;
        cin>>a>>b;
        if(b==0)
            throw "Division by negative number";
        res+=a/b;
        display();
    }
    void display(){
        cout<<res<<endl;
    }
};
```

Exception in Class

```
int main(){
    Calculator c;
    try{
        c.division();
    }catch(const char *e){
        cout<<e<<endl;
    }
    return 0;
}
```

Object as Exception

```
#include<iostream>
#include<string>
using namespace std;

class MyException{
    string exceptionText;
    public:
    MyException(string text){
        exceptionText=text;
    }
    string what(){
        return exceptionText;
    }
};
```

Object as Exception

```
int main(){
    int a;
    cout<<"Enter any integer"<<endl;
    cin>>a;
    try{
        if(a<0)
            throw MyException("Negative Number");
    }catch(MyException e){
        cout<<e.what()<<endl;
    }
    return 0;
}
```

Terminology

- **Exceptions** are run-time anomalies that a program may detect
  - division by 0,
  - access to an array outside of its bounds,
  - exhaustion of the heap memory.
- An exception is an unusual event, and may require special processing
- The special processing required after detection of an exception is called **exception handling**
- The exception handling code unit is called an **exception handler**

Introduction to Exception Handling

- Traditional approaches in case of exception:
  - Terminate the program: `exit()`, `abort()`
  - Return special values to indicate errors in a function
  - Set global error bits and return normally (leaving the system in an illegal state)
- Error detection and handling code is tedious to write; it clutters the program and is error-prone
- The C++ language provides <u>built-in features</u> to raise and handle exceptions:
  - Separate error-handling code from ordinary code
  - Exception propagation allows a high level of reuse of exception handling code
  - Release local resources automatically

What is exception handling?

- It provides a mechanism to decouple handling of errors or other exceptional circumstances from the typical control flow of our code.
- Provides more freedom to handle errors.
- Enable returning codes cause whenever required.

Basics of exception handling

- Using three keywords that works in conjunction with each other: **throw, try, and catch**.
- In C++, a **throw** statement is used to signal that an exception or error case has occurred.
  - It is also commonly called raising an exception.
- Using a **throw** statement
  - throw *val;*
- Typically, this value will be an error code, a description of the problem, or a custom exception class.

General Form

```
try {
  -- code that is expected to raise an
  exception
}
// Each catches one type of exception
catch (type1 var1) {        // var is optional
  -- handler code
}
...
catch (type2 var2) {
  -- handler code
}
```

An Example

```
#include <iostream>
using namespace std;

int main () {
   try {
     throw 20;   // throw an exception
   } catch (int e) {
     cout << "Exception No. " << e << endl;
   }
   return 0;
}
```

try

- Put statements and function calls that may generate exceptions in a `try` block
- Each try block is associated with a sequence of handlers that follow immediately
- `try` blocks can be *nested*

```
try {
  try {
   f();   // f() may throw an exception
  } catch (int e) {
   cout << "Exception No. " << e << endl;
  }
} catch(double) { cout << "Caught double." << endl; }
```

throw

- An exception is raised using a **throw expression**, composed of `throw` followed by an object whose type is that of the exception thrown
- Any object (built-in or user-defined) can be thrown

```
class to_be_thrown {};
...
throw to_be_thrown; // error, not an object
throw to_be_thrown(); // correct
throw 2.5;            // correct, double
```

`catch`: The Handler

- `catch` is the name of all **handlers**
  - must immediately follow the try block
  - the formal parameter of each handler must be *unique*
  - no automatic type conversion
- The formal parameter does not have to be a variable
  - Can be simply a type name to distinguish its handler from others
  - A variable transfers information to the handler

`catch`: The Handler

- The formal parameter can be an <u>ellipsis</u>, in which case it handles all exceptions not yet handled

```
catch (...) {   // catches everything
      -- handler code
}
```

- After a handler completes, control flows to the first statement after the last handler in the sequence
- When no exception occurs, all handlers are neglected (no performance loss)

Propagation/Stack Unwinding

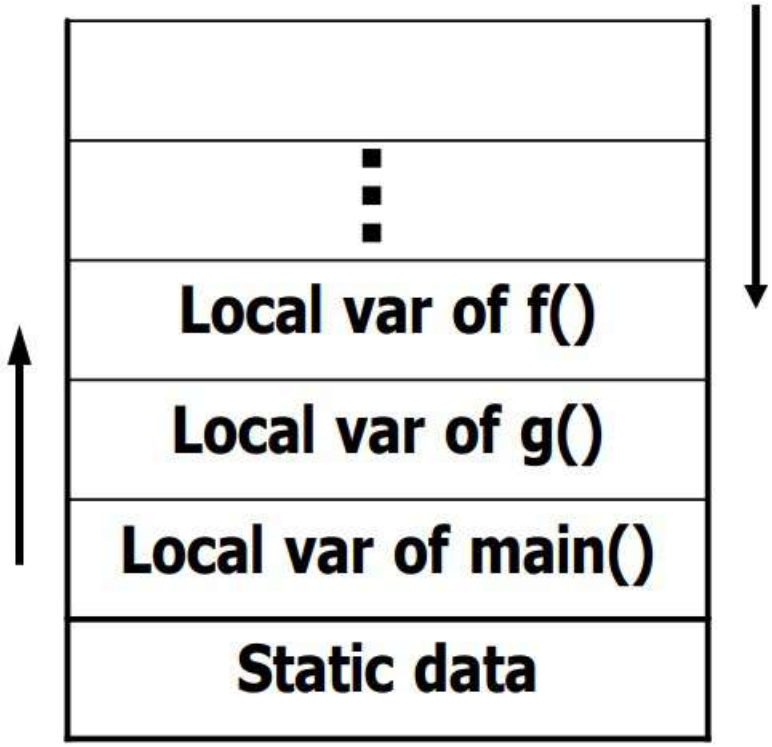
- Exceptions CANNOT be ignored
- If not caught by handlers right after `try` block, exception moves to next-higher level and may be caught there:
  - The next level of try block (if nested)
  - Try block surrounding the function call in which exception occurs
  - If no handler at any level catches the exception, `terminate()` will be called and program will terminate
- Passing an exception while searching for a handler can cause abnormal exit from a function while in middle of executing it (i.e., without any return value)
  - The stack frame corresponding to the exited function's scope is popped – this is called stack unwinding
  - So the lifetime of local objects in the exited functions ends
  - C++ still guarantees correct destructors are called

An Example

```
void f() {
  Person p;
  throw 20;
}
void g() {Person g; f();}
void main(){
  try {
    g();
  } catch(int) {
    cout<<"error"<<endl;
  }
}
```

call function

Unwind



Snapshot of Memory Stack

throw Examples

- // throw a literal integer value
  - throw -1;
- // throw an enum value
  - throw ENUM_INVALID_INDEX;
- // throw a literal char* string
  - throw "Can not take square root of negative number";
- // throw a double variable that was previously defined
  - throw dX;
- // Throw an object of class MyException
  - throw MyException("Fatal Error");
- Each of these statements acts a signal that some kind of problem that needs to be handled has occurred.

Handling exceptions

- Handling exceptions is actually the job of the catch block(s).
- The **catch** keyword is used to define a block of code (called a catch block) that handles exceptions for a single data type.
- Try blocks and catch blocks work together
  - A try block detects any exceptions that are thrown by statements within the try block, and routes them to the appropriate catch block for handling.
- A try block must have at least one catch block attached to it, but may have multiple catch blocks.

```
catch (int) {
    // Handle an exception of type int here
    cerr << "We caught an exception of type int" << endl;
}
```

```
int main(){
    try {
        // Statements that may throw
      //exceptions you want to handle now go here
        throw -1;
    }
    catch (int){
    // Any exceptions of type int thrown
   //within the above try block get sent here
        cerr << "We caught an exception of type int" << endl;
    }
    catch (double){
      // Any exceptions of type double thrown
      //within the above try block get sent here
        cerr << "We caught an exception of type double" << endl;
    }
     return 0;
}
```

Output:

We caught an exception of type int

How exception works?

- When an exception is raised (using throw), execution of the program immediately jumps to the nearest enclosing try block.
- If any of the catch handlers attached to the try block handle that type of exception, that handler is executed.
- If no appropriate catch handlers exist, execution of the program propagates to the next enclosing try block.
- If no appropriate catch handlers can be found before the end of the program, the program will fail with an exception error.

Example

```
int main() {
    try {
        throw 4.5; // throw exception of type double
        cout << "This never prints" << endl;
    }
    catch(double dX) // handle exception of type double {
        cerr << "We caught a double of value: " << dX << endl;
    }
}
```

Output:

We caught a double of value: 4.5

Example(2)

```
int main() {
    cout << "Enter a number: ";
    double dX;
    cin >> dX;
    try  {
      // If the user entered a negative number
      //, this is an error condition
        if (dX < 0.0)
            throw "Can not take sqrt of negative number";
            cout << "The sqrt of " << dX << " is " << sqrt(dX) << endl;
   }
   catch (char* strException) {
       cerr << "Error: " << strException << endl;
   }
}
```

Nested try blocks

```
int main ()
{
    try
    {
        try
        {
            throw 1.0;
        }
        catch (int x)
        {
            cout << "Exception int type";
        }
    }
    catch (double x)
    {
        cout << "Exception double type";
    }
}// end of main
```

Exception inside a function

```
double MySqrt(double dX) {
    if (dX < 0.0)
        throw "Can not take sqrt of negative number";
     // throw exception of type char*
    return sqrt(dX);
}
int main()  {
    cout << "Enter a number: ";
    double dX;
    cin >> dX;
    try {
        cout << "The sqrt of " << dX << " is " << MySqrt(dX) << endl;
    }
    catch (const char* strException) // catch exceptions of type char* {
        cerr << "Error: " << strException << endl;
    }
}
```

Uncaught exception

```
double MySqrt(double dX){
        if (dX < 0.0)
        throw "Can not take sqrt of negative number";
        return sqrt(dX);
}
 int main() {
    cout << "Enter a number: ";
    double dX;
    cin >> dX;
     // Look, no exception handler!
    cout << "The sqrt of " << dX << " is " << MySqrt(dX) << endl;
}
```

Explanation

- When a function doesn't handle the exception, so the program stack unwinds and control returns to the function where it is called.
- If there's no exception handler here either, it returns to previous function and this continues till it reaches main function().
- If main() either dont have a handler, it terminates.
- When main() terminates with an unhandled exception, the operating system will generally notify us that an unhandled exception error has occurred.
- It depends on the operating system how to handle this error
- Possibilities
  - printing an error message
  - popping up an error dialog
  - Simply crashing. (avoid this case)

catch all handler

- C++ provides us with a mechanism to catch all types of exceptions.
- This is known as a catch-all handler.
- A catch-all handler works just like a normal catch block, except that instead of using a specific type to catch, it uses the ellipses operator (…) as the type to catch.

```
try {
    throw 5.5; // throw an int exception
}
catch (double dX){
    cout << "We caught an exception of type double: " << dX << endl;
}
catch (...) // catch-all handler {
    cout << "We caught an exception of an undetermined type" << endl;
}
```

catch all handler

- The catch-all handler should be placed last in the catch block chain.
- This is to ensure that exceptions can be caught by exception handlers tailored to specific data types if those handlers exist.
- Often, the catch-all handler block is left empty:
  - catch(...) {} // ignore any unanticipated exceptions

terminate() & unexpected()

- Throwing an unhandled exception causes the standard library function **terminate()** to be invoked.
- Attempting to throw an exception that is not supported by a function will cause the standard library function **unexpected( )** to be called.
- By default, terminate( ) and unexpected() calls **abort()** to stop your program, but you can specify your own termination handler